2.5.2 Loops

Procedures let you give sequences of *Mathematica* operations to execute. You also often need to give operations that are to be executed repetitively, in some kind of "loop".

`Do[`*expr*`, {`*i*`, `*imax*`}]`	evaluate *expr* repetitively, with *i* varying from `1` to *imax* in steps of `1`
`Do[`*expr*`, {`*i*`, `*imin*`, `*imax*`, `*istep*`}]`	evaluate *expr* with *i* varying from *imin* to *imax* in steps of *istep*
`Do[`*expr*`, {`*n*`}]`	evaluate *expr* *n* times
`Nest[`*f*`, `*expr*`, `*n*`]`	apply *f* to *expr* *n* times
`FixedPoint[`*f*`, `*expr*`]`	start with *expr*, and apply *f* repeatedly until the result no longer changes
`While[`*test*`, `*expr*`]`	evaluate *expr* repetitively, so long as *test* is `True`
`For[`*start*`, `*test*`, `*step*`, `*expr*`]`	evaluate *start*, then repetitively evaluate *step* and *expr*, until *test* fails

Loop constructs.

This makes a loop with `i` running from 1 to 4. It prints the first four squares.

```
In[1]:= Do[Print[i^2], {i, 4}]

1
4
9
16
```

This makes a loop with `k` running from 2 to 4.

```
In[2]:= t = x; Do[t = 1/(1 + k t), {k, 2, 4}]; t

                 1
Out[2]= ---------------
                 4
        1 + -----------
                   3
            1 + -------
                1 + 2 x
```

Here `k` runs from 0 to 5 in steps of 2.

```
In[3]:= r = 1; Do[r = (r + k)/(1 + k), {k, 0, 5, 2}]; r

Out[3]= 1
```

The way iteration is specified in `Do` is exactly the same as in functions like `Table` and `Sum`. Just as in those functions, you can set up several nested loops by giving a sequence of iteration specifications to `Do`.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; http://www.wolfram.com/; 1-800-441-6284.

This loops over values of i from 1 to 4, and for each value of i, loops over j from 1 to i-1.

```
In[4]:= Do[Print[{i,j}], {i, 4}, {j, i-1}]
{2, 1}
{3, 1}
{3, 2}
{4, 1}
{4, 2}
{4, 3}
```

One important point to remember is that, as in other iteration functions, the iteration variable in Do cannot have a value before the iteration starts. You can use Block to make sure that the iteration variable in Do is kept local.

The iteration variable cannot already have a value before the iteration starts.

```
In[5]:= m = 3; Do[Print[m^2], {m, 3}]
General::itervar:
    In iterator {m, 3}, variable m already has a value.

                    2
Out[5]= Do[Print[m ], {m, 3}]
```

The Block makes m local, so that it can be used in the Do.

```
In[6]:= Block[{m}, Do[Print[m^2], {m, 3}]]
1
4
9
```

Sometimes you may want to repeat a particular operation a certain number of times, without changing the value of an iteration variable. You can specify this kind of repetition in Do just as you can in Table and other iteration functions.

This repeats the assignment t = 1/(1+t) three times.

```
In[7]:= t = x; Do[t = 1/(1+t), {3}]; t
```

$$Out[7]= \frac{1}{1 + \frac{1}{1 + \frac{1}{1 + x}}}$$

You can put a procedure inside Do.

```
In[8]:= t = 67; Do[Print[t]; t = Floor[t/2], {3}]
67
33
16
```

Do allows you to repeat an operation many times. Often what you actually want to do is just to take an expression, and repeatedly apply the same function to it.

Nest applies a function repeatedly.

```
In[9]:= Nest[f, x, 3]
Out[9]= f[f[f[x]]]
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; http://www.wolfram.com/; 1-800-441-6284.

This applies a pure function three times, giving the same result as the Do in the example above.

```
In[10]:= Nest[ Function[t, 1/(1+t)], x, 3 ]

                    1
Out[10]= ----------------
                      1
          1 + ------------
                        1
               1 + -------
                    1 + x
```

Nest allows you to apply a function a specified number of times. In some cases, you may simply want to go on applying a function until the results you get no longer change. You will need to do this if you want to imitate evaluation in *Mathematica*, or the operation of the //. operator. FixedPoint goes on applying a function until it gets the same result on two successive occasions.

FixedPoint goes on applying a function until the result no longer changes.

```
In[11]:= FixedPoint[Function[t, Print[t];Floor[t/2]], 67]

67
33
16
8
4
2
1
0

Out[11]= 0
```

The function Do allows you to set up a loop in which iteration variables take on a fixed sequence of values. *Mathematica* also provides the somewhat more flexible iteration functions For and While, which allow you to perform a sequence of operations repeatedly, stopping when some arbitrary condition fails to be true.

The While loop continues until the condition fails.

```
In[12]:= n = 17; While[(n = Floor[n/2]) != 0, Print[n]]

8
4
2
1
```

This is a very common form for a For loop. i++ increments the value of i.

```
In[13]:= For[i=1, i < 4, i++, Print[i]]

1
2
3
```

Here is a more complicated For loop. Notice that the loop terminates as soon as the test i^2 < 10 fails. The body of the loop is not executed the last time through.

```
In[14]:= For[i=1; t=x, i^2 < 10, i++, t = t^2 + i;
                 Print[t]]

         2
1 + x

           2 2
2 + (1 + x )

                 2 2 2
3 + (2 + (1 + x ) )
```

The way `For` and `While` work in *Mathematica* is similar to the way they work in the C programming language. If you are familiar with C, however, you should be sure to notice that the roles of comma and semicolon are reversed in *Mathematica* `For` loops relative to C ones.

When you set up loops in *Mathematica*, particularly with `For` and `While`, you often want to update repeatedly the values of variables you are using. There are short ways to specify some of the more common operations of this kind.

i`++`	increment the value of *i* by `1`
i`--`	decrement *i*
`++`*i*	pre-increment *i*
`--`*i*	pre-decrement *i*
i `+=` *di*	add *di* to the value of *i*
i `-=` *di*	subtract *di* from *i*
x `*=` *c*	multiply *x* by *c*
x `/=` *c*	divide *x* by *c*
`{`*x*`, `*y*`} = {`*y*`, `*x*`}`	interchange the values of *x* and *y*
`PrependTo[`*list*`, `*elem*`]`	prepend *elem* to the value of *list*
`AppendTo[`*list*`, `*elem*`]`	append *elem*

Some operations that are often used in loops.

This sets `t` to `x`, then increments `t` by the symbolic quantity `5y`, then returns the result for `t`.

```
In[15]:= t = x; t += 5y; t
Out[15]= x + 5 y
```

The value of `i++` is the value of `i` *before* the increment is done.

```
In[16]:= i=5; Print[i++]; Print[i]
5
6
```

The value of `++i` is the value of `i` *after* the increment.

```
In[17]:= i=5; Print[++i]; Print[i]
6
6
```

The assignment `{a, b} = {b, a}` interchanges the values of the variables `a` and `b`.

```
In[18]:= (a=1; b=2; Print[{a,b}]; {a, b} = {b, a};
          Print[{a,b}])

{1, 2}
{2, 1}
```

You can make assignments of lists to permute values of variables in any way.

```
In[19]:= (a=1; b=2; c=3; Print[{a,b,c}];
                    {a, b, c} = {c, a, b}; Print[{a,b,c}])

{1, 2, 3}
{3, 1, 2}
```

When you set up loops in *Mathematica*, you will often need to take a list, and successively add elements to it. The functions `PrependTo` and `AppendTo` allow you to manipulate lists "by name".

`PrependTo[v, x]` is equivalent to `v = Prepend[v, x]`.

```
In[20]:= v = {a, b, c}; PrependTo[v, x]; v
Out[20]= {x, 3, 1, 2}
```